

R: limpieza, manipulación y exploración de datos

Cheatsheet de pivots en R

Jonatán Perren

12 de mayo de 2026

Las funciones de pivot transforman la **forma** de un data frame:

- `pivot_longer()` — de **ancho** a **largo**: colapsa varias columnas en filas.
- `pivot_wider()` — de **largo** a **ancho**: expande filas en columnas.

Ambas forman parte del paquete **tidyr** (incluido en el tidyverse).

El concepto subyacente es el de **datos ordenados** (tidy data): una variable por columna, una observación por fila, un valor por celda.

Datos anchos vs. datos largos

Formato ancho

país	2021	2022	2023
ARG	480	630	620
URY	50	58	62

Una fila por país. Cada año es una columna.

Formato largo

país	año	pbi
ARG	2021	480
ARG	2022	630
ARG	2023	620
URY	2021	50
URY	2022	58
URY	2023	62

Una fila por combinación país-año.

El formato largo es el estándar tidy: facilita el filtrado, la agregación y la visualización con `ggplot2`.

Datasets usados en este machete

Todos pertenecen al paquete `tidyr` — disponibles con `library(tidyr)` o `library(tidyverse)`, sin instalar nada extra.

Dataset	Descripción
billboard	ranking Billboard Hot 100 (año 2000): <code>artist</code> , <code>track</code> , <code>date.entered</code> , <code>wk1-wk76</code>
relig_income	distribución de ingresos por religión (Pew Research): <code>religion</code> + 10 tramos de ingreso
who2	casos de tuberculosis OMS: <code>country</code> , <code>year</code> + columnas con forma <code>diagnóstico_sexo_edad</code>
us_rent_income	ingreso mediano y alquiler por estado de EEUU (2017 ACS): <code>GEOID</code> , <code>NAME</code> , <code>variable</code> , <code>estimate</code> , <code>moe</code>
fish_encounters	avistamientos de peces en estaciones de monitoreo: <code>fish</code> , <code>station</code> , <code>seen</code>
world_bank_pop	población mundial Banco Mundial: <code>country</code> , <code>indicator</code> , columnas 2000-2017

`anscombe` (cuarteto de Anscombe) es de base R: disponible sin ningún `library()`.

¿Para qué sirve `pivot_longer()`?

En `billboard`, las semanas son columnas (`wk1`, `wk2`, ..., `wk76`): no existe una columna única semana. `ggplot2` no puede graficar `aes(x = semana, y = posicion)`.

Con `pivot_longer()` los datos quedan en formato tidy:

```
1 billboard ▶  
2   pivot_longer(starts_with("wk"),  
3     names_to = "semana", values_to = "posicion") ▶  
4   ggplot(aes(x = semana, y = posicion, group = track)) +  
5     geom_line()
```

También habilita `group_by(semana) ▶ summarise()` y `filter(semana == "wk10")` sobre una sola columna.

pivot_longer() – caso mínimo

```
1 library(tidyr)
2
3 relig_income > pivot_longer(
4   cols       = !religion, # todas menos "religion"
5   names_to   = "ingreso",
6   values_to  = "n"
7 )
```

```
# A tibble: 180 x 3
  religion ingreso      n
  <chr>      <chr>   <int>
1 Agnostic <$10k         27
2 Agnostic $10-20k      34
3 Agnostic $20-30k      60
```

`relig_income` tiene 10 columnas de tramos de ingreso. `!religion` selecciona todas las columnas excepto `religion`.

pivot_longer() – selección de columnas

Patrón de cols	Selecciona
<code>cols = 2:5</code>	columnas por posición (2 a 5)
<code>cols = c(ene, feb, mar)</code>	columnas nombradas
<code>cols = starts_with("wk")</code>	columnas cuyo nombre empieza con <code>wk</code>
<code>cols = where(is.numeric)</code>	todas las columnas numéricas
<code>cols = !religion</code>	todas excepto <code>religion</code>
<code>cols = everything()</code>	todas las columnas

Se admiten todos los tidy-select helpers: `ends_with()`, `matches()`, `num_range()`, etc.

pivot_longer() – names_prefix

Elimina un prefijo fijo de los nombres de columna al convertirlos en valores.

```
1 billboard > pivot_longer(  
2   cols      = starts_with("wk"),  
3   names_to  = "semana",  
4   names_prefix = "wk",  
5   values_to = "posicion"  
6 )
```

	artist	track	date.entered	semana	posicion
	<chr>	<chr>	<date>	<chr>	<dbl>
1 2	Pac	Baby Don't Cry (Ke)	2000-02-26	1	87
2 2	Pac	Baby Don't Cry (Ke)	2000-02-26	2	82

names_prefix acepta una expresión regular. El prefijo se elimina siempre del inicio del nombre.

pivot_longer() – names_transform

La columna de claves es **character** por defecto. **names_transform** cambia su tipo.

```
1 billboard > pivot_longer(  
2   cols      = starts_with("wk"),  
3   names_to  = "semana",  
4   names_prefix = "wk",  
5   names_transform = list(semana = as.integer),  
6   values_to  = "posicion"  
7 )
```

	artist	track	date.entered	semana	posicion
	<chr>	<chr>	<date>	<int>	<dbl>
1 2	Pac	Baby Don't Cry	2000-02-26	1	87

Se pueden transformar varias claves a la vez:

```
list(semana = as.integer, otra = as.factor).
```

pivot_longer() – values_drop_na

Descarta automáticamente las filas donde el valor pivotado es **NA**.

```
1 billboard > pivot_longer(  
2   cols      = starts_with("wk"),  
3   names_to  = "semana",  
4   values_to = "posicion",  
5   values_drop_na = TRUE  
6 )
```

En **billboard**, la mayoría de canciones no dura 76 semanas en el ranking: las semanas sin posición son **NA**. Sin **values_drop_na = TRUE** el resultado conserva esas filas vacías.

Equivale a agregar **> filter(!is.na(posicion))** después del pivot, pero más eficiente.

pivot_longer() – múltiples claves en names_to

Cuando el nombre de columna codifica **varias variables**: cada parte se convierte en una columna de clave.

```
1 who2 > pivot_longer(  
2   cols      = !c(country, year),  
3   names_to  = c("diagnostico", "sexo", "edad"),  
4   names_sep = "_"  
5 )
```

	country	year	diagnostico	sexo	edad	count
	<chr>	<int>	<chr>	<chr>	<chr>	<int>
1	Afghanistan	1980	sp	m	014	NA
2	Afghanistan	1980	sp	m	1524	NA

Argumento	Cuándo usarlo
names_sep	el separador es un carácter fijo (_, ., -)
names_pattern	el separador varía o no existe; se necesita regex

pivot_longer() — el sentinel ".value"

Si una parte del nombre de columna es el **nombre de una variable** (no una categoría), se usa **".value"**: esa parte se convierte en nombre de columna, no en valor de celda.

anscombe contiene 4 conjuntos de datos artificiales con las mismas estadísticas (media, varianza, correlación) pero patrones visuales completamente distintos.

Sin ".value":

conjunto	variable	valor
1	x	10.0
1	y	8.04
2	x	10.0
2	y	9.14

Con ".value":

conjunto	x	y
1	10.0	8.04
2	10.0	9.14
3	10.0	7.46
4	8.0	6.58

```
1 anscombe ► pivot_longer(  
2   cols           = everything(),  
3   names_to      = c(".value", "conjunto"),  
4   names_pattern = "([xy])([1-4])"  
5 )
```

¿Para qué sirve `pivot_wider()`?

En `us_rent_income`, `income` y `rent` son filas de la columna `variable`: no existen como columnas. Calcular `mutate(income / rent)` produce un error en tiempo de ejecución.

Con `pivot_wider()` cada categoría pasa a ser una columna:

```
1 us_rent_income ►  
2   pivot_wider(names_from = variable, values_from = estimate) ►  
3   mutate(ratio = income / rent)
```

	GEOID	NAME	income	rent	ratio
	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	01	Alabama	24476	747	32.8
2	02	Alaska	32940	1200	27.5

pivot_wider() – caso mínimo

```
1 us_rent_income ► pivot_wider(  
2   names_from = variable, # valores -> nombres de columna  
3   values_from = estimate # valores -> celdas  
4 )
```

	GEOID	NAME	income	rent
	<chr>	<chr>	<dbl>	<dbl>
1	01	Alabama	24476	747
2	02	Alaska	32940	1200
3	04	Arizona	27517	972

pivot_wider() – values_fill

Cuando una combinación identificador × nombre no existe, la celda es **NA**. **values_fill** la reemplaza.

```
1 fish_encounters > pivot_wider(  
2   names_from = station,  
3   values_from = seen,  
4   values_fill = 0  
5 )
```

values_fill acepta un escalar (aplica a todas las columnas nuevas) o una lista nombrada para distintos valores por columna: **values_fill = list(seen = 0)**.

pivot_wider() – names_prefix

Agrega un prefijo a los nombres de las columnas creadas.

```
1 us_rent_income ► pivot_wider(  
2   names_from = variable,  
3   names_prefix = "est_",  
4   values_from = estimate  
5 )
```

	GEOID	NAME	est_income	est_rent
	<chr>	<chr>	<dbl>	<dbl>
1	01	Alabama	24476	747

Útil cuando `names_from` contiene números: evita columnas con nombres que empiezan con dígitos (que requieren backticks en R).

pivot_wider() – values_fn

Cuando hay más de un valor para la misma celda, `values_fn` define cómo agregarlos.

```
1 datos > pivot_wider(  
2   names_from = categoria,  
3   values_from = monto,  
4   values_fn   = sum  
5 )
```

Sin `values_fn`, los duplicados generan columnas de tipo `lista` (`<list>`). Si aparece ese tipo en el resultado, hay filas duplicadas o falta `values_fn`.

pivot_wider() – id_cols

Por defecto, `pivot_wider` usa todas las columnas que no son `names_from` ni `values_from` como identificadores de fila. `id_cols` permite especificarlos explícitamente.

```
1 us_rent_income > pivot_wider(  
2   id_cols      = c(GEOID, NAME),  
3   names_from  = variable,  
4   values_from = estimate  
5 )
```

Sin `id_cols`, la columna `moe` también actúa como identificador involuntario de las columnas de tipo `<list>`. `id_cols` aísla exactamente qué columnas identifican cada fila.

pivot_wider() – múltiples columnas de valores

`values_from` acepta un vector: genera una columna por combinación nombre × variable.

```
1 us_rent_income ► pivot_wider(  
2   names_from = variable,  
3   values_from = c(estimate, moe)  
4 )
```

	GEOID	NAME	estimate_income	estimate_rent	moe_income	moe_rent
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	01	Alabama	24476	747	136	3

El orden es: todos los valores de `names_from` para la primera variable de `values_from`, luego para la segunda.

pivot_wider() – names_glue

Controla el patrón de nombres de columna cuando `values_from` recibe un vector. La cadena se interpola con `{.value}` (la variable) y el nombre de la columna de `names_from`.

```
1 us_rent_income > pivot_wider(  
2   names_from = variable,  
3   values_from = c(estimate, moe),  
4   names_glue = "{variable}_{.value}"  
5 )
```

	GEOID	NAME	income_estimate	income_moe	rent_estimate	rent_moe
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	01	Alabama	24476	136	747	3

Sin `names_glue`: `estimate_income`, `estimate_rent`, `moe_income`, `moe_rent` – la variable va primero, luego el valor de `names_from`.

El ciclo largo ↔ ancho

```
1 pop_largo ← world_bank_pop ▷
2   pivot_longer(
3     cols      = !c(country, indicator),
4     names_to  = "year",
5     values_to = "value"
6   )
7
8 pop_largo ▷
9   pivot_wider(
10    names_from = year,
11    values_from = value
12  )
```

El ciclo es reversible mientras no se pierda información: sin `values_drop_na = TRUE` ni `values_fn`.

Argumento	Uso
<code>cols</code>	columnas a colapsar (tidy-select)
<code>names_to</code>	nombre(s) de la nueva columna de claves
<code>values_to</code>	nombre de la nueva columna de valores
<code>names_prefix</code>	prefijo a eliminar de los nombres de columna
<code>names_transform</code>	lista para convertir el tipo de las claves
<code>values_drop_na</code>	si TRUE, descarta filas con valor NA
<code>names_sep</code>	carácter que separa variables en el nombre de columna
<code>names_pattern</code>	regex con grupos de captura (alternativa a <code>names_sep</code>)

Argumento	Uso
<code>names_from</code>	columna cuyos valores se convierten en nombres de columna
<code>values_from</code>	columna(s) cuyos valores llenan las celdas
<code>values_fill</code>	valor para celdas sin datos (NA por defecto)
<code>names_prefix</code>	prefijo a agregar a los nuevos nombres de columna
<code>names_glue</code>	patrón de interpolación; usa <code>{.value}</code> y el valor de <code>names_from</code>
<code>values_fn</code>	función de agregación cuando hay duplicados
<code>id_cols</code>	columnas que identifican cada fila (por defecto: el resto)